

Basic ? Crackme

dimanche 5 juillet 2026 14:20

```

126: int main (int argc, char **argv);
|  ~: args(rdi, rsi) vars(4:sp[0x10..0x88])
|
|  0x00001302  55      push rbp
|  0x00001303  4889e5    mov rbp, rsp
|  0x00001306  4883c480  add rsp, 0xffffffff80
|  0x0000130a  897d8c    mov dword [var_74h], edi ; argc
|  0x0000130d  48897580  mov qword [var_80h], rsi ; argv
|  0x00001311  64488b0425.. mov rax, qword fs:[0x28]
|  0x0000131a  488945f8  mov qword [canary], rax
|  0x0000131e  31c0     xor eax, eax
|  0x00001320  b800000000 mov eax, 0
|  0x00001325  e895ffff   call sym.init
|  0x0000132a  488d05f30c.. lea rax, str.Key: ; 0x2024 ; "Key: "
|  0x00001331  4889c7    mov rdi, rax ; const char *format
|  0x00001334  b800000000 mov eax, 0
|  0x00001339  e832fdffff call sym.imp.printf ; int printf(const char *format)
|  0x0000133e  488d4590  lea rax, [var_70h]
|  0x00001342  4889c6    mov rsi, rax
|  0x00001345  488d05de0c.. lea rax, [0x0000202a] ; "%s"
|  0x0000134c  4889c7    mov rdi, rax ; const char *format
|  0x0000134f  b800000000 mov eax, 0
|  0x00001354  e827fdffff call sym.imp.__isoc99_scanf ; int scanf(const char *format)
|  0x00001359  488d4590  lea rax, [var_70h]
|  0x0000135d  4889c7    mov rdi, rax ; char *arg1
|  0x00001360  e8a6feffff call sym.check
|  0x00001365  b800000000 mov eax, 0
|  0x0000136a  488b55f8  mov rdx, qword [canary]
|  0x0000136e  64482b1425.. sub rdx, qword fs:[0x28]
|  jrc 0x00001377 7405     je 0x137e
|  | 0x00001379 e8d2fcffff call sym.imp.__stack_chk_fail ; void __stack_chk_fail(void)
|  | ; CODE XREF from main @ 0x1377(x)
|  ↳ 0x0000137e c9      leave
|  0x0000137f c3      ret

```

```

.data:00000000000004048 key db 0A8h ; DATA XREF: check+3C ↑ o
.data:00000000000004049 db 96h
.data:0000000000000404A db 4Fh ; O
.data:0000000000000404B db 7Fh ;
.data:0000000000000404C db 3Eh ; >
.data:0000000000000404D db 94h
.data:0000000000000404E db 0Ah
.data:0000000000000404F db 95h
.data:0000000000000404F _data ends
.data:0000000000000404F

```

Perso j'ai codé ce code qui fait d'abord un xor avec le masque et la clé de base pour ensuite prendre cette nouvelle clé et faire un autre xor avec. Ce qui nous donne la vrai clé pour résoudre ce challenge.

```

#include<stdio.h>
#include <stdlib.h>
#include <string.h>
/*
key db 0A8h
.data:00000000000004049 db 96h
.data:0000000000000404A db 4Fh ; O
.data:0000000000000404B db 7Fh ;
.data:0000000000000404C db 3Eh ; >
.data:0000000000000404D db 94h
.data:0000000000000404E db 0Ah
.data:0000000000000404F db 95h
.data:0000000000000404F _data ends
*/

//nouvelle_key[i] = key[i] XOR masque[i]

//0xA377AD570465FDF9

int main (){
unsigned char magic[8] = {0xF9, 0xFD, 0x65, 0x04, 0x57, 0xAD, 0x77, 0xA3};

unsigned char masque[] = {0x19, 0x01, 0x19, 0x01, 0x19, 0x01, 0x19, 0x01};
unsigned char var14 [30] = {0};

```

```

/*for(int i = 0; i < sizeof(key); i++){
    res[i] = magic[i] ^ key[i] % 8;
}
*/

for(int x = 0; x < sizeof(key); x++){
    newkey[x] = key[x] ^ masque[x];
    res[x] = magic[x] ^ newkey[x % 8];
}

for(int i = 0; i < sizeof(key); i++){
    res[i] = magic[i] ^ newkey[i % 8];
}

for(int z = 0; z < sizeof(key); z++){
    printf("%c",res[z]);
}

puts("\n");

for(int w = 0; w < sizeof(key); w++){
    printf("%c",res1[w]);
}

return 0;
}

```

2 of 3

qu'elles soient définies quelque part, et le sont donc dans cette fonction, systématiquement appelée à chaque exécution du binaire. Une fois décompilée, on s'aperçoit que son code n'est pas ce à quoi on pourrait s'attendre et qu'il apparaît très suspect :

```
1. signed __int64 __fastcall do_global_ctors_aux()
2. {
3.     unsigned __int64 v0; // r10
4.     signed __int64 result; // rax
5.     int i; // [rsp+0h] [rbp-10h]
6.     __QWORD *v3; // [rsp+4h] [rbp-8h]
7.
8.     result = sys_ptrace(0LL, 0LL, 0LL, v0);
9.     result = (unsigned int)result;
10.    if ( (_DWORD)result != -1 )
11.    {
12.        result = (signed __int64)&data_start;
13.        v3 = &data_start;
14.        for ( i = 0; i <= 255; ++i )
15.        {
16.            result = 0x950A943E7F4F96A8LL;
17.            if ( *v3 == 0x950A943E7F4F96A8LL )
18.            {
19.                result = (signed __int64)v3;
20.                *v3 = 0x0119011901190119LL;
21.            }
22.        }
23.    }
24.}
```

On y retrouve, dans un premier temps, l'appel à **ptrace**, explicitant donc l'ensemble des instructions qui sont mises de côté à l'exécution du programme débuggé... On y lit aussi que lorsque le programme est exécuté de manière classique, une tranche de sa mémoire, commençant au symbole **data_start** et ayant une taille de 2040 octets, est parcourue par extraits de 8 octets (un QWORD). Ce parcours est fait à la recherche de la valeur **0x950A943E7F4F96A8**. Dès qu'une d'elle est trouvée, elle se retrouve xorée par **0x0119011901190119** et le programme poursuit son cours.

Cette fonction a pour conséquence que la donnée **key**, justement égale à **0x950A943E7F4F96A8**, utilisée dans la fonction **check** n'est plus la même qu'au départ. Il suffit donc de la corriger et d'effectuer la même manipulation qu'initialement pour tomber sur une chaîne qui valide le programme non tracé !

Merci beaucoup pour le challenge ! :)